Machine Learning II

Data Science M.Sc.

Prof. Dr. Steffen Wagner
Berliner Hochschule für Technik

Winter term 2025/26

Gradient Boosting

Why?

Kaggle AI Report 2023: Tabular Data (p. 38)

Trends & Predictions

There are three main trends with machine learning for tabular data:

1. *Need for unique approaches for every dataset/problem.*
2. *Outsized importance of data munging and feature engineering.*
3. *Continuing dominance of **gradient boosted trees** as the algorithm of choice.*

The first two trends have been well known in the Kaggle circles ever since the platform launched, and the last since XGBoost was first introduced on Kaggle in 2014 (you can find a collection of winning Kaggle solutions that use XGBoost here.

`https://www.kaggle.com/AI-Report-2023`

Introduction

Two Main Categories: **Ensemble Learning**

1. **Bagging** (Bootstrap Aggregating):
   - Trains models **independently** on *bootstrapped* subsets of data (observations and features).
   - Focuses on **Variance Reduction** by averaging (e.g., Random Forest).
2. **Boosting**:
   - Trains models **sequentially**, where each new model is focused on correcting the errors of the preceding ensemble.
   - Each model is a **weak learner** only being slightly better than random chance (typically a shallow decision tree, a 'stump').
   - Focuses on **Bias Reduction** (e.g., Gradient Boosting).

Outline

Content:

1. The Core Concept:
   - Boosting and base learners
   - Gradient Boosting and Pseudo-Residuals
2. Different Loss functions and their applicability
3. Gradient tree boosting: Regression Trees as base learners of choice
4. Recent Developments:
   - Discussion of modern libraries like **XGBoost, LightGBM, and CatBoost**
   - Focusing on key features like advanced regularization and performance optimizations.

Goal:

- Understanding why Gradient Boosting is a **powerful** and **highly flexible** optimization framework.
- Learning the details of the most popular and advanced implementations.

Gradient Boosting Machines

The Concept

The Additive Model

Boosting builds a final predictor $F(x)$ by summing up the predictions of M weak learners $h_m(x)$:

$$F_M(x) = \sum_{m=1}^{M} \gamma_m h_m(x)$$

where γ_m are weights or scaling factors.

Stage-Wise Construction

The model is built *sequentially* one step at a time:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$$

- Initialisation: $F_0(x) = \overline{y}$
- At stage m, we seek the weak learner h_m that, when added to the current ensemble F_{m-1}, best reduces the overall loss.
- The **loss** (or objective) is a function

$$L(F_m, y)$$

  that quantifies the the penalty for incorrect predictions.
- In Boosting the loss is based on the previous predictor function $F_{m-1}(x)$ and the newly added weak learner $\gamma_m h_m(x)$.

The Objective

We want to minimize the loss L so that the newly added weak learner is optimal:

$$\begin{aligned}\hat{h}_m, \hat{\gamma}_m &= \underset{h_m, \gamma_m}{\operatorname{argmin}} L\left(F_m(x), y\right) \\ &= \underset{h_m, \gamma_m}{\operatorname{argmin}} L\left(F_{m-1}(x) + \gamma_m h_m(x), y\right)\end{aligned}$$

Important:

- The choice of the loss functions determines, if you want to perform Classification, mean, robust or quantile regression (see table on slide 39 for details).
- L can be also a user-defined, *differentiable* loss function for very specific optimization tasks.
- Solving this optimization problem will lead to so-called pseudo-residuals $\tilde{y}_i$, showing *where* the model is still wrong and needs further improvement.

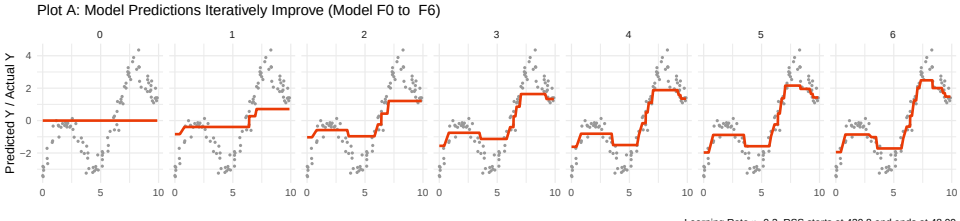
Boosting: an illustrative example

The Analogy: The Contractor and the Blueprints

Imagine you are managing a construction project:

GBM Component	Analogy	Description
y **(True Value)**	The **Architect's Blueprint** (The perfect house plan).	What the model *should* predict.
$F_{m-1}(x)$ **(Prediction)**	The **Current Structure** (The house built so far).	What the model *currently* predicts.
L **(Loss Function)**	The **Inspection Check-list** (The metric for quantifying failure).	How badly the current structure misses the blueprint.
Pseudo-Residual ($\tilde{y}_i$)	The **Punch List** (The list of mandatory fixes).	The ideal corrective action to minimize the next inspection's penalty.

Stage-Wise Construction: example



The Optimization Challenge

- We are looking for an optimal function $\hat{h}_m(x)$ and optimal γ_m that minimize the loss:

$$\hat{h}_m, \hat{\gamma}_m = \underset{h_m, \gamma_m}{\operatorname{argmin}} L\left(F_{m-1}(x) + \gamma_m h_m(x), y\right)$$

  This is done in a two sequential steps:
  1. Estimation of $\hat{h}_m$.
  2. Determination of $\hat{\gamma}_m$.
- The greedy optimization over the *entire function space* in step 1 is computationally infeasible.
- **Gradient Boosting** re-frames the step 1 problem as **Gradient Descent** in the function space.

The Functional Gradient ($\nabla_F L$)

The functional gradient is the generalization of the vector gradient to the space of functions.

1. **Definition:** The functional gradient is the derivative of the loss L with respect to the prediction function $F(x)$:
$$\nabla_F L(F) \equiv \frac{\partial L(y, F(x))}{\partial F(x)}$$
2. **Interpretation:** At any data point x_i, the functional gradient gives the direction of the *greatest increase* in the loss.
3. Looking at the **negative functional gradient** gives the direction of the **steepest descent**.
It tells us: *"What target output should the next function $h_m(x_i)$ predict to maximize the immediate decrease in loss L?"*

Pseudo-Residuals

In GBM, we don't calculate the function $\nabla_F L$ directly. We calculate its value at each training point x_i resulting in N-dimensional vector of **Pseudo-Residuals**

$$\tilde{y}_i = \tilde{y}_{i,m} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}$$

Interpretation:

- The pseudo-residuals show us to what extent the prediction function $F(x)$ for feature value x_i has to be corrected to minimize the overall loss L as direct as possible.
- If the sum of the pseudo-residuals is zero, the minimum of $L(y, F(x))$ is found.
- For Squared-Error-Loss the pseudo-residuals are exactly the standard residuals you are used to (see following slides).

Example 1: **Squared Error Loss**

Simple Regression Task: modeling a continuous target variable y.

- **Current Model:** $F_{m-1}(x)$ is the prediction from the ensemble of all previous trees.
- **Goal:** Calculate the pseudo-residual $\tilde{y}_i$ for a single data point (x_i, y_i).
- **Loss Function:** Squared Error Loss

$$L(y, F) = \frac{1}{2}(y - F)^2$$

remark: the factor of $\frac{1}{2}$ is added to the standard Squared Error Loss function for mathematical convenience when taking the derivative, not for changing the objective of the loss itself

- **Example Data Point**

Variable	Description	Value
y_i	True Value (Target)	**10.5**
$F_{m-1}(x_i)$	Current Model Prediction	**8.4**

Step 1: Calculate the Functional Gradient ($\nabla_F L$)

The functional gradient is the derivative of the loss L with respect to the prediction $F(x)$:

$$\nabla_F L(y_i, F(x_i)) = \frac{\partial}{\partial F}\left[\frac{1}{2}(y_i - F(x_i))^2\right]$$

Using the chain rule:

$$\nabla_F L = \frac{1}{2} \cdot 2 \cdot (y_i - F(x_i)) \cdot (-1) = -(y_i - F(x_i)) = F(x_i) - y_i$$

Calculation for our example: with $F(x_i) = F_{m-1}(x_i)$

$$\nabla_F L = 8.4 - 10.5 = -\mathbf{2.1}$$

Step 2: Calculate the Pseudo-Residual ($\tilde{y}_i$)

The pseudo-residual is the **negative** of the functional gradient. This is the "target" the next weak learner $h_m(x)$ will try to predict.

$$\tilde{y}_i = -\nabla_F L$$

Calculation for our example:

$$\tilde{y}_i = -(-2.1) = \mathbf{2.1}$$

Interpretation: Pseudo-Residual = Standard Residual

For Squared Error Loss, the pseudo-residual $\tilde{y}_i$ is exactly the **standard residual**

$$\tilde{y}_i = y_i - F_{m-1}(x_i).$$

- The current model predicted 8.4, but the true value is 10.5. The error is $10.5 - 8.4 = 2.1$.
- The next weak learner $h_m(x)$ is trained to predict this residual/remaining error ($\tilde{y}_i = 2.1$) for feature value x_i to correct the current model's output and move $F_m(x)$ closer to y_i.

– end of example 1–

Example 2: **Absolute Error Loss**

Robust Regression Task: modelling a continuous target variable y

- **Current Model:** $F_{m-1}(x)$ is the prediction from the ensemble of all previous trees.
- **Goal:** Calculate the pseudo-residual $\tilde{y}_i$ for a single data point (x_i, y_i).
- **Loss Function:** Absolute Error Loss

$$L(y, F) = |y - F|$$

- **Example Data Point**

Variable	Description	Value
y_i	True Value (Target)	**10.5**
$F_{m-1}(x_i)$	Current Model Prediction	**8.4**

Steps 1&2: Functional Gradient ($\nabla_F L$) and Pseudo-Residual $\tilde{y}_i$

The functional gradient is the derivative of the loss L with respect to the prediction $F(x)$:

$$\nabla_F L(y_i, F(x_i)) = \frac{\partial}{\partial F} |y_i - F(x_i)|$$

Using the chain rule:

$$\nabla_F L = \text{sign}(F - y)$$

Calculation for our example: with $F(x_i) = F_{m-1}(x_i)$

$$\nabla_F L = \text{sign}(8.4 - 10.5) = \text{sign}(-2.1) = -\mathbf{1} \quad \text{and} \quad \tilde{y}_i = -\nabla_F L = \mathbf{1}$$

– end of example 2–

Learning h_m from Pseudo-Residuals ($\tilde{y}_i$) for General Loss L

- The process of finding the optimal next weak learner h_m, involves replacing the complex, non-linear optimization dictated by L with a simple, standard regression problem.
- $\tilde{y}_i$ is a numerical value that represents the exact, necessary **correction** at point x_i to move the model in the direction of steepest loss reduction.
- $\tilde{y}_i$ is now the **new target variable** for the next weak learner h_m to be estimated from data.

The Learning Step (The Squared Error Fit)

The algorithm finds the function h_m that best approximates the new target variable $\tilde{y}_i$. The standard and computationally simplest way to do this is by minimizing the **Squared Error** between the new target $\tilde{y}_i$ and the prediction $h_m(x_i)$:

$$\hat{h}_m = \underset{h}{\operatorname{argmin}} \sum_{i=1}^{N} (\tilde{y}_{i,m} - h(x_i))^2$$

An **important point** in Gradient Boosting is, that h_m is *always* learned using the **Squared Error Loss against the pseudo-residuals** $\tilde{y}_i$ regardless of the original loss function L.

Why Squared Error?

- Applying the squared error criteria at this stage gives us the **weak learner** $\hat{h}_m$ that is **most highly correlated with** the **theoretically and unknown** h_m based on the data distribution leading to $\tilde{y}_{i,m}$.

  (Friedman, Jerome H. "Greedy function approximation: a gradient boosting machine." Annals of statistics (2001): 1189-1232. `https://doi.org/10.1214/aos/1013203451`)
- Bonus: regression is designed to solve squared error problems efficiently. By using $\tilde{y}_i$ as the target, the problem of deriving h_m becomes a standard, well-solved regression task, even if the original loss L was highly complex (like Huber loss or Quantile loss).

What we did so far:

We applied an elegant dual-objective approach to estimate $\hat{h}_m$ as solution to the Gradient Boosting task:

Phase	Goal	Mechanism	Key Output
1. Direction	Define the ideal direction of steepest loss reduction.	Calculate the **Negative Functional Gradient** (the derivative of the loss L w.r.t. the prediction F).	**Pseudo-Residuals ($\tilde{y}_i$)**
2. Structure	Find the best base learner h_m to approximate that ideal direction.	Train **any regression model** $h_m(x)$ (Linear, Spline, NN, or Tree) to minimize **Squared Error** against the target $\tilde{y}_i$.	**Fixed function** $h_m(x)$

Important: The function h_m only cares about approximating the numeric values of $\tilde{y}_i$ using the standard Squared Error criterion. It does not need to know or understand the complex formula of the original loss L.

What's next?

Goal: We are looking for an optimal function $h_m(x)$ **and** an optimal scaling factor γ_m that minimizes the loss:

$$\hat{h}_m, \hat{\gamma}_m = \underset{h_m, \gamma_m}{\operatorname{argmin}} L\left(F_{m-1}(x) + \gamma_m h_m(x), y\right)$$

Q: Are we close to this goal?

A: Yes, we already estimated h_m. Now we have to determine the optimal factor γ_m.

Determining the Optimal Step Size γ_m

The final step is to find the single best coefficient γ_m that scales the previously estimated $\hat{h}_m(x)$ to maximally minimize the **original loss function L** across the entire dataset.

$$\gamma_m = \underset{\gamma}{\operatorname{argmin}} \sum_{i=1}^{N} L\left(y_i, F_{m-1}(x_i) + \gamma \cdot \hat{h}_m(x_i)\right)$$

This is typically solved using **Numerical Line Search techniques** (like Brent's Method) or by using the **Newton-Raphson approximation** (if L is twice-differentiable) to find the optimal γ_m.

Summary: so far

Task: Finding $\hat{h}_m, \hat{\gamma}_m$ via $\underset{h_m, \gamma_m}{\operatorname{argmin}} L\left(F_{m-1}(x) + \gamma_m h_m(x), y\right)$

Solution:

Phase	Goal	Mechanism	Key Output
1. Direction	Define the ideal direction of steepest loss reduction.	Calculate the **Negative Functional Gradient** (the derivative of the loss L w.r.t. the prediction F).	**Pseudo-Residuals ($\tilde{y}_i$)**
2. Structure	Find the best base learner h_m to approximate that ideal direction.	Train **any regression model** $h_m(x)$ (Linear, Spline, NN, or Tree) to minimize **Squared Error** against the target $\tilde{y}_i$.	**Fixed function** $h_m(x)$
3. Magnitude	Find the optimal step size along the fixed direction $h_m(x)$.	Perform **Numerical Line Search** on the **Original Loss** L to find the scalar γ_m.	**Optimal Coefficient** γ_m

Final Model Update

Once all three steps are complete, the model is updated. The final model prediction is the sum of the previous ensemble and the new scaled step:

$$F_m(x) = F_{m-1}(x) + \hat{\gamma}_m \hat{h}_m(x)$$

- $\hat{h}_m$ is the optimal weak learner.
- $\hat{\gamma}_m$ is the **optimal coefficient**.

Final Model Update

Once all three steps are complete, the model is updated. The final model prediction is the sum of the previous ensemble and the new scaled step:

$$F_m(x) = F_{m-1}(x) + \hat{\gamma}_m \hat{h}_m(x)$$

- $\hat{h}_m$ is the optimal weak learner.
- $\hat{\gamma}_m$ is the **optimal coefficient**.

Hey, wait a moment. This is happening too fast!

Gradient Boosting Machines

Regularization

Controlling the Pace

The optimal coefficient $\hat{\gamma}_m$ (derived in Phase 3) represents the theoretically perfect step size along the direction $\hat{h}_m$.
However, taking a full, optimal step at every iteration is often not ideal because it might lead to **Overfitting**.

How to avoid overfitting?

- By taking smaller steps, i.e. we shrink $\hat{\gamma}_m$ by an additional factor $\eta < 1$, the model $F(x)$ is forced to combine information from *many* update iterations (M must be large).
- No single update step dominates the prediction, making the final ensemble smoother and less sensitive to noise.

Final Model Update including Regularization

The final model prediction is the sum of the previous ensemble and the new, scaled and **regularized** step:

$$F_m(x) = F_{m-1}(x) + \eta \cdot \hat{\gamma}_m \hat{h}_m(x)$$

- $\hat{h}_m$ is the optimal weak learner.
- $\hat{\gamma}_m$ is the **optimal coefficient**.
- η (Learning Rate) is a **hyperparameter** used to regularize (shrink) the step. η is a factor typically $0.001 < \eta \leq 1$.

The Learning Rate

Value of η	Effect on Learning	Resulting Model
$\eta = 1.0$	Full step is taken. Fast convergence.	High risk of **overfitting** (high variance).
$\eta = 0.1$ (Standard)	A fraction of the step is taken. Slow learning.	**Better generalization** (low variance).
$\eta \approx 0.01$	Very small steps taken. Very slow convergence.	Often leads to the **most accurate, robust** models.

Further regularization needed

- Even if we successfully constrain the weak learners h_m to be really weak (e.g., tree with max. depth of 3), they will all be **trained on the same data**.
- This leads to correlated pseudo-residuals and therefore **correlated $\mathbf{h_m}$** counteracting the shrinkage based on η by taking many small (due to η) and identical steps leading in sum again to the large step we wanted to avoid.
- This means that in addition to the shrinkage of $\gamma_m \to \eta \cdot \gamma_m$, we have to break the correlations between the individual h_m.
- The standard approach to limit these correlation between the ensemble's weak learners is **Stochastic Gradient Boosting**[1].

[1] Friedman, *Stochastic gradient boosting*, 2002 https://doi.org/10.1016/S0167-9473(01)00065-2

Stochastic Gradient Boosting (SGB)

- By training h_m on a random subset of observations, each tree sees a slightly different view of the data.
- When the ensemble averages these slightly different (decorrelated) predictions, the overall variance of the final model decreases, leading to better out-of-sample performance.
- Therefore SGB falls under *structural regularization* because it limits the model's capacity to overfit the training data by modifying the training process itself.
- **Bonus:** Estimating h_m on a subset (between 50% and 70% of the observations) speeds up the model training.

The Different Types of Regularization

Regularization Technique	Primary Target	Mechanism	Reason for Necessity
Weak Learner Constraints (e.g., trees with Max Depth 3)	**High Variance in** h_m	Restricts the model's **complexity** and **fitting power** by limiting splits.	Prevents *individual* trees from over-relying on local noise.
Learning Rate (η)	**Overfitting Due to Aggressive Learning**	Shrinks the **step size** of the ensemble update.	Ensures learning is slow and gradual, acting as a final L_2-style regularizer for γ_m.
Stochasticity (SGB/Subsampling)	**Correlation Between Trees**	Fits each tree h_m on a **random subset** of observations.	**Decorrelates** the trees. Even simple trees trained on the same data tend to have correlated errors, which SGB breaks up, leading to better generalization.

Wrapping all up …

GBM on one slide !

GBM Optimization framework (with SGD) that builds an additive model $F_M(x)$ by performing **Steepest Descent in Function Space**: $F_m(x) = F_{m-1}(x) + \eta \cdot \hat{\gamma}_m \hat{h}_m(x)$

Phase	Goal	Mechanism	Key Output
1. Direction	Define the ideal direction of steepest loss reduction.	Calculate the **Negative Functional Gradient** (the derivative of the loss L w.r.t. the prediction F).	**Pseudo-Residuals ($\tilde{y}_i$)**
2. Structure	Find the best base learner h_m to approximate that ideal direction using a **random sub-sample** (e.g., 70% of the observations).	Train **any regression model** $h_m(x)$ (Linear, Spline, NN, or Tree) to minimize **Squared Error** against the target $\tilde{y}_i$	**Fixed function** $h_m(x)$
3. Magnitude	Find the optimal step size along the fixed direction $h_m(x)$.	Perform **Numerical Line Search** on the **Original Loss** L to find the scalar γ_m.	**Optimal Coefficient** γ_m
4. Regularization	Control the learning pace and prevent overfitting.	Apply the **Learning Rate (η)** to shrink the final step $\eta \cdot \gamma_m$.	**Final Update** $F_m(x)$

Loss functions

Overview: Loss functions and Use Cases

Remember: The **choice of the loss function L defines the task** (e.g. regression, classification, ...) you want to perform!

Name	Loss function L	Pseudo residual $\tilde{y}_i = -\frac{\partial L}{\partial F}$	Use case
Squared Error (L_2 norm)	$L_2 = \frac{1}{2}(y - F)^2$	$\tilde{y}_i = y - F$	Standard **Regression** (Minimize MSE)
Absolute Error (L_1 norm)	$L_1 = \lvert y - F \rvert$	$\tilde{y}_i = \text{sign}(y - F)$	**Robust Regression** (Minimize MAE)
Log Loss (Binary Deviance)	$L_{\text{Log}} = \log(1 + e^{-yF})$ with $y \in \{-1, +1\}$	$\tilde{y}_i = \frac{y}{1+e^{yF}}$	Standard **Binary Classification**
Pinball / Quantile	$L_\tau = \begin{cases} \tau(y - F) & \text{if } y > F \\ (1 - \tau)(F - y) & \text{if } y \leq F \end{cases}$	$\tilde{y}_i = \begin{cases} \tau & \text{if } y > F \\ \tau - 1 & \text{if } y < F \end{cases}$	**Quantile Regression** (Predict percentile τ)
Huber Loss	$L_{\text{Huber}} = \begin{cases} \frac{1}{2}(y - F)^2 & \text{for } \lvert y - F \rvert \leq \delta \\ \delta(\lvert y - F \rvert - \frac{1}{2}\delta) & \text{for } \lvert y - F \rvert > \delta \end{cases}$	$\tilde{y}_i = \begin{cases} y - F & \text{for } \lvert y - F \rvert \leq \delta \\ \delta \cdot \text{sign}(y - F) & \text{for } \lvert y - F \rvert > \delta \end{cases}$	**Robust Regression** (Smooth L1/L2 blend)

Gradient tree boosting

- Up to now, we didn't restrict h_m to a specific model class.
- From now on, we discuss Gradient Tree Boosting which implies that h_m is a regression tree.

Decision trees are overwhelmingly preferred as weak learners in Gradient Boosting for three main reasons:

1. Mathematical Convenience
2. Model Simplicity (Weakness)
3. Feature Handling

Mathematical Convenience using trees

The primary reason trees are ideal is that they simplify the optimization required by the Gradient Boosting framework:

- **Least Squares Efficiency:** The inner loop of GBM requires the weak learner h_m to fit the pseudo-residuals $\tilde{y}_i$ by minimizing the **Squared Error**. Regression trees are designed to be extremely efficient at minimizing the Squared Error for piecewise constant functions.
- **Per-Leaf Optimization:** A regression tree partitions the feature space into distinct, non-overlapping regions (leaves, $R_{m,j}$). This localization allows the subsequent optimization for the optimal step size, $\gamma_{m,j}$, to be solved **independently and efficiently** within each leaf:
  For any leaf region $R_{m,j}$ found by the regression tree h_m, the optimal value for the step size $\gamma_{m,j}$ has only to be determined based on the observations in this leaf.

Model Simplicity

Boosting requires **weak learners**, i.e. models that are simple and have high bias.

Trees are easy to constrain to be weak:

- **Controllable Complexity:** By setting a small **maximum depth** (e.g., depth 1 to 6), trees can be explicitly made simple and prone to high bias. This ensures they only capture a small portion of the overall pattern (the negative gradient) at each step.
- **Additive Power:** When hundreds or thousands of these simple, high-bias models are added together sequentially, the combined model becomes highly complex and achieves low bias and low variance, realizing the full power of boosting.

Flexibility and Data Handling

Trees handle complex, real-world data structures very well, making them robust base learners:

- **Non-Linearity & Interactions:** Trees naturally model complex **non-linear relationships** and **feature interactions** without requiring the user to manually engineer these interactions.
- **Invariant to Scaling:** Trees are not sensitive to the scaling of features (e.g. standardization is not required).
- **Mixed Data Types:** Trees inherently handle both **numerical and categorical features** simultaneously.
- Robust handling of **Missing Data** (Surrogate Splits).

Hyperparameter optimization: Tuning the Model

Hyperparameter optimization (HPO)

- Hyperparameters are external settings that govern the entire training process, while internal parameters (or model parameters) are the values the learning algorithm itself adjusts based on the training data.
- Hyperparameter optimization (HPO) is the process of finding the set of external configuration settings for a machine learning algorithm—such as the learning rate (η) or maximum tree depth—that yields the best model performance on a validation dataset.

Hyperparameters for Gradient Boosted Trees

Hyperparameter	Description	Typical Impact	Best Practice
M **(n_estimators)**	Number of sequential trees.	Higher M increases performance but risks overfitting.	Use **Early Stopping** based on a validation set.
η **(Learning Rate)**	Shrinkage factor for each new tree's contribution.	Smaller η means a slower, more robust model, requiring larger M.	Start with $0.01 - 0.1$. **Crucial for performance.**
Max Depth	Depth of the base regression trees.	Limits complexity; low depth (e.g., 3-7) is common as trees are kept weak.	Prevents individual trees from overfitting; tune via cross-validation.
Subsampling	Fraction of data used to train each tree (Stochastic GB).	Introduces randomness, reduces variance, speeds up training.	Recommended: $0.5 - 0.8$.

Setting up HPO

HPO is performed in practice using a systematic search strategy to find the configuration that minimizes the loss function on a held-out **validation set**. Since the search space can be vast, the goal is often to find a good-enough solution efficiently, rather than the absolute mathematical optimum.

Before any search begins, two things must be defined:

1. **Hyperparameter Space:** Define the range of values for each hyperparameter (e.g., η between 0.01 and 0.3, max. tree depth between 3 and 10).
2. **Objective Function:** The metric to be minimized (e.g., cross-validation error or Validation AUC). This metric can differ from the specified loss function.

HPO strategies

Common HPO Search Strategies:

- **Grid Search:** Evaluates the model at every point on a manually defined grid of hyperparameter values which makes it computationally expensive. The cost grows exponentially with the number of hyperparameters.
- **Random Search:** Samples hyperparameter combinations randomly from the defined space. Often finds better results faster than Grid Search because it explores the high-dimensional space more effectively. It's often enough to find a good configuration quickly.
- **Bayesian Optimization:** Uses the results of previous evaluations to intelligently choose the next best set of parameters to try. Requires significantly fewer function evaluations than Grid or Random Search to find a good minimum.

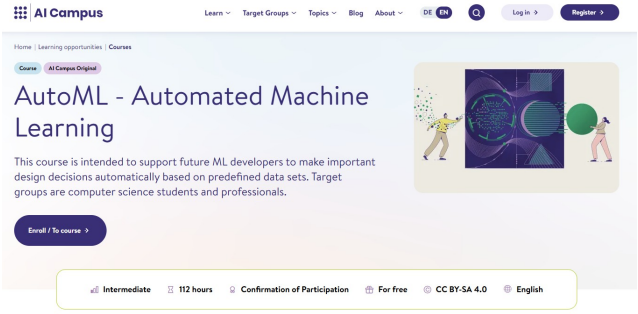
HPO performance

To make HPO practical, efficiency techniques are crucial:

- **Cross-Validation (CV):** Instead of a simple train/validation split, **k-fold cross-validation** is typically used to obtain a more robust estimate of the model's performance for a given configuration.
- **Early Stopping:** For iterative algorithms like Gradient Boosting, the optimization can be halted early if the validation loss has not improved for a set number of epochs. This saves time and computational resources.
- **Hyperband/ASHA (Adaptive Resource Allocation):** These advanced methods quickly discard poor hyperparameter configurations after only a few epochs of training, saving resources for the more promising candidates.

HPO: further reading

If you want to learn more about HPO as being an essential part of automated machine learning (Auto-ML), a very good source is the following (free of charge) online course:



https://ki-campus.org/en/learning-opportunities/courses/automl-automated-machine-learning

State-of-the-Art Gradient Boosting

More than ten years of longing …

The core mathematical theory was sound in 2001, but the engineering challenge of making it fast and scalable required significant breakthroughs.

In essence, Friedman provided the engine (GBM), but the SOTA libraries provided the necessary high-performance transmission, fuel injection, and aerodynamics to make it a race car.

pictures taken from: `https://statistics.stanford.edu/people/jerome-h-friedman`; `https://tqchen.com/`; `https://de.wikipedia.org/wiki/XGBoost#/media/Datei:XGBoost_logo.svg`

The Computational Bottleneck of GBM

Friedman's original Gradient Boosting Machine (GBM), while mathematically correct, was inherently a **sequential, CPU-bound algorithm**. This created a significant bottleneck:

- **Inherent Sequentiality:** Boosting is designed to be sequential (Tree m depends on Tree $m-1$). The original algorithm could not be easily parallelized across multiple cores or machines, making training slow for large datasets.
- **Memory Management:** The GBM process requires a new dependent variable $\tilde{y}_i$ in every single iteration that is dense and non-static leading to a huge amount of I/O operations. As datasets grew in the late 2000s and early 2010s, this hardware requirement became prohibitive for standard hardware.
- **Inefficient Split Finding:** The standard process for finding the optimal split in a decision tree involves sorting features, which is computationally expensive and was not optimized in early GBM implementations.

The SOTA Breakthroughs

Implementation	Innovation	Solution Provided
XGBoost (Chen, 2014)	Optimized Tree Construction (Approximate Split)	Introduced **Approximate Split Finding** (using quantiles) and **Column Subsampling** to significantly speed up the tree construction phase.
	Second-Order Approximation (Advanced Step Size)	Leveraged **Second-Order Taylor Expansion of L** to derive closed-form, highly accurate loss minimization formulas for leaf weights γ_m **and** split criteria simultaneously. This is a **genuine breakthrough** in both speed (no numerical search needed) and accuracy **finding the optimal loss-related tree strcuture h_m**.
	System Optimization & Parallelization	Implemented parallelization for *independent tasks* (like finding the best split for each feature) across CPU cores and even GPUs, overcoming the sequential training bottleneck.
LightGBM (Microsoft, 2017)	Advanced Data Handling (GOSS & EFB)	Introduced **Gradient-based One-Side Sampling (GOSS)** and **Exclusive Feature Bundling (EFB)** to drastically reduce the number of samples and features needed to calculate splits, leading to massive speed increases on large data.
CatBoost (Yandex, 2017)	Categorical Feature Handling & Robustness	Introduced **Ordered Boosting** and **Ordered Target Encoding** to natively handle categorical features and mitigate target leakage bias for better robustness.

Deep Dive: **XGBoost 2nd-Order Approximation**

Recap: **Friedmans** two-step process for building the update $\gamma_m h_m$:

1. Find tree structure h_m:
   - Fit a standard regression tree to the pseudo-residuals $\tilde{y}_i$
   - Split Criterion: **Minimize L2 Error (MSE) for** $\tilde{y}_i$
   - **But:** fitting $\tilde{y}_i$ is not the same as minimizing L.
2. Find Leaf Values γ_m:
   - *After* the tree is built: a numerical line search is performed in each leaf
   - This is an **iterative, slow** process.

Deep Dive: **XGBoost 2nd-Order Approximation**

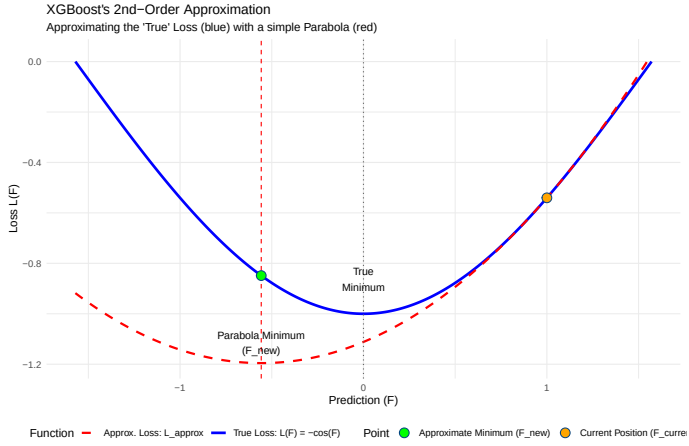
XGBoost: A **single, unified** process using a quadratic closed form approximation of L

Only one step required: Finding the tree structure h_m (= split values) **and** Leaf Values γ_m simultaneously, in a way that *directly* maximizes the reduction in the *Loss* L.

How it works:

1. The algorithm calculates the minimum of the quadratic approximation to obtain optimal γ_m^* for any *potential* split value.
2. Additionally, the *potential* "Gain" (= decrease in loss L) for each *potential* split for optimal γ_m^* is calculated.
3. The final tree h_m only consists of splits directly maximizing gain directly adressing the true objective, i.e. minimizing the loss L.

Deep Dive: **XGBoost 2nd-Order Approximation**



Deep Dive: **XGBoost 2nd-Order Approximation**

Key Takeaways:

- **Accuracy:** XGBoost is more accurate because its split criterion (**Gain**) is directly derived from the *true* loss function L, not a proxy the $\tilde{y}_i$.
- **Efficiency:** It's also faster because the optimal γ_m is calculated *directly* from the closed-form quadratic approximation of L. This is much faster than the *slow* numerical search.

Gradient-based One-Side Sampling (GOSS)

GOSS samples the data asymmetrically to preserve the informative samples (= still showing large pseudo-residuals) while randomly discarding the data points already fitted quite well.

1. *Selection of Set A (The Hard Samples):* All samples whose gradients (= pseudo-residuals) are in the top a% (where a is a small constant, e.g., 30%) are *kept*.
2. *Selection of Set B (The Easy Samples):* A small random subset of the remaining samples (Set B, the small-gradient ones) is *kept*. The majority of Set B is discarded.
3. **Weighting:** The samples kept from the small-gradient set (Set B) are *up-weighted*. This compensates for the discarded samples, ensuring the new tree's objective function remains an unbiased estimate of the full gradient distribution.

Advantage:
GOSS significantly *speeds up training* by reducing the effective dataset size (N) used in tree construction while *maintaining high accuracy* because it avoids discarding the critical, error-prone data points.

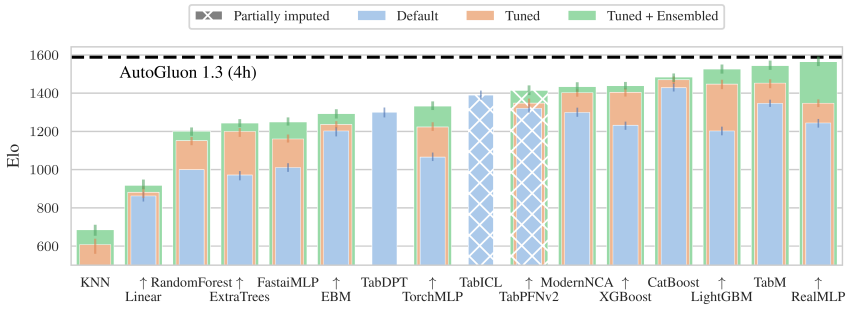
Literature

All details can be found in

- Chen, T., & Guestrin, C. (2016, August). **Xgboost**: A scalable tree boosting system. In Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining (pp. 785-794). https://doi.org/10.48550/arXiv.1603.02754
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., … & Liu, T. Y. (2017). **Lightgbm**: A highly efficient gradient boosting decision tree. Advances in neural information processing systems, 30. https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). **CatBoost**: unbiased boosting with categorical features. Advances in neural information processing systems, 31. https://proceedings.neurips.cc/paper/2018/hash/14491b756b3a51daac41c24863285549-Abstract.html

Performance: predictive accuracy

TabArena Elo ranking (October 23rd, 2025):



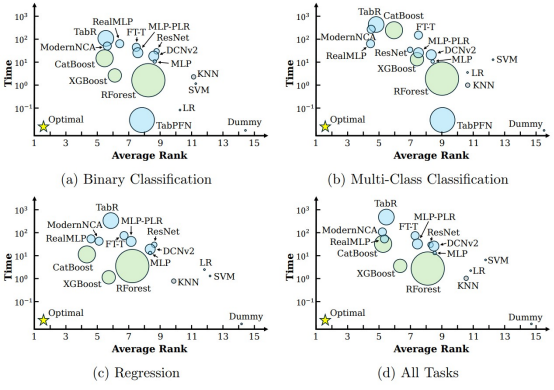
sources:

real time benchmarking: `https://huggingface.co/spaces/TabArena/leaderboard`

Erickson, N., Purucker, L., Tschalzev, A., Holzmüller, D., Desai, P. M., Salinas, D., & Hutter, F. (2025). Tabarena: A living benchmark for machine learning on tabular data. arXiv preprint arXiv:2506.16791. `https://doi.org/10.48550/arXiv.2506.16791`

Performance: predictive accuracy vs. computational costs vs. task

Important: Performance is a high-dimensional construct



(a) Binary Classification (b) Multi-Class Classification

(c) Regression (d) All Tasks

source: Ye, H. J., Liu, S. Y., Cai, H. R., Zhou, Q. L., & Zhan, D. C. (2024). A closer look at deep learning methods on tabular datasets. arXiv preprint arXiv:2407.00956. `https://arxiv.org/pdf/2407.00956`

How to choose the optimal implementation?

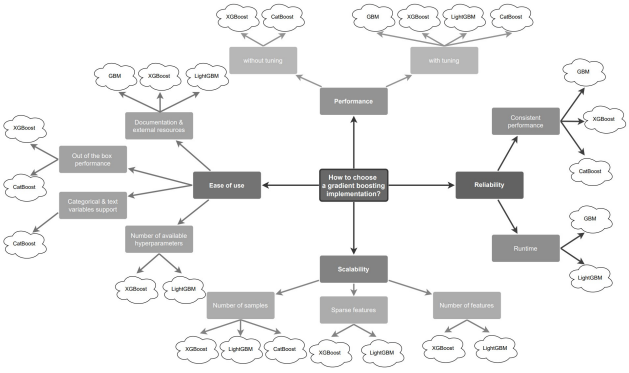


Figure 6: The choice of optimal gradient boosting implementation in different scenarios

Florek, P., & Grabinski, M. (2023). Benchmarking state-of-the-art gradient boosting algorithms for classification. arXiv preprint arXiv:2305.17094
https://doi.org/10.48550/arXiv.2305.17094

Conclusion and Summary

- **Gradient Boosting** is a powerful ensemble technique that sequentially builds an additive model.
- It generalizes boosting by using a **Gradient Descent** approach in function space.
- The key is training weak learners (regression trees) to fit the **negative gradient** (pseudo-residuals) of the loss function.
- Only the **choice of the loss function** determines the task, i.e. regression or classification, which makes GMB a very general framework.
- **Hyperparameter Tuning**, especially the **Learning Rate** and **Number of Trees**, is critical to balance bias and variance.
- Modern implementations like **XGBoost**, **LightGBM** and **CatBoost** offer high performance and sophisticated regularization.
- It is not crystal clear which SOTA implementation is best, so the implementation can be considered being another hyperparameter …